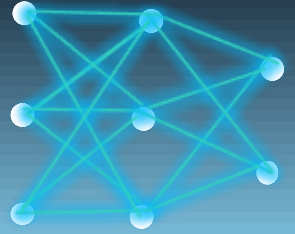




Deep Learning

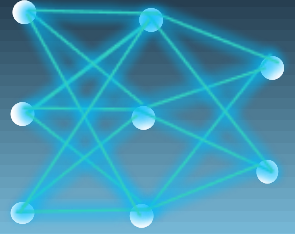
Dr. Shay Maymon

Convolutional Neural Networks (CNNs)

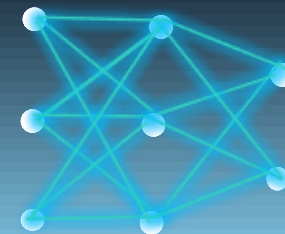


- CNNs are designed for structured data (especially images)
- **Key idea:**
 - Sparse connectivity
 - Parameter sharing across space
 - Designed to encode invariances and equivariances specific to image data

Four Key Inductive Biases in CNNs



- CNNs exploit four coupled concepts:
 - **Hierarchy**
 - **Locality**
 - **Equivariance**
 - **Invariance**

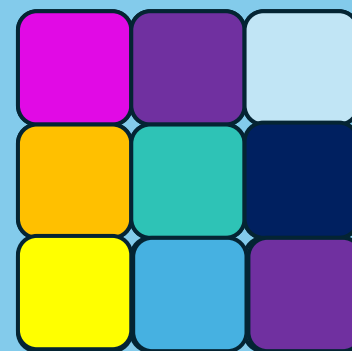
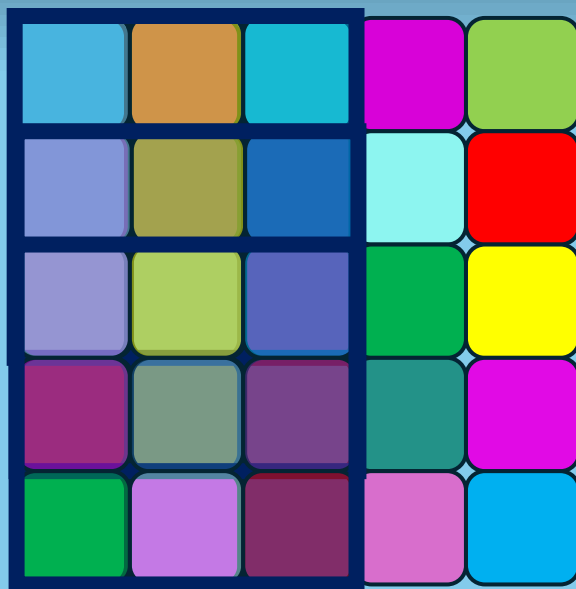
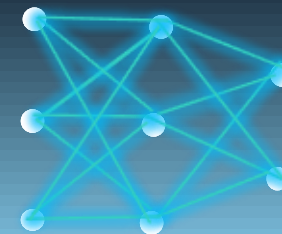


Mathematical Form of Convolution

- For an image I with pixel intensities $I(j, k)$ and a filter K with pixel values $K(l, m)$, the feature map $C = I * K$ (ignoring bias and nonlinear activation):

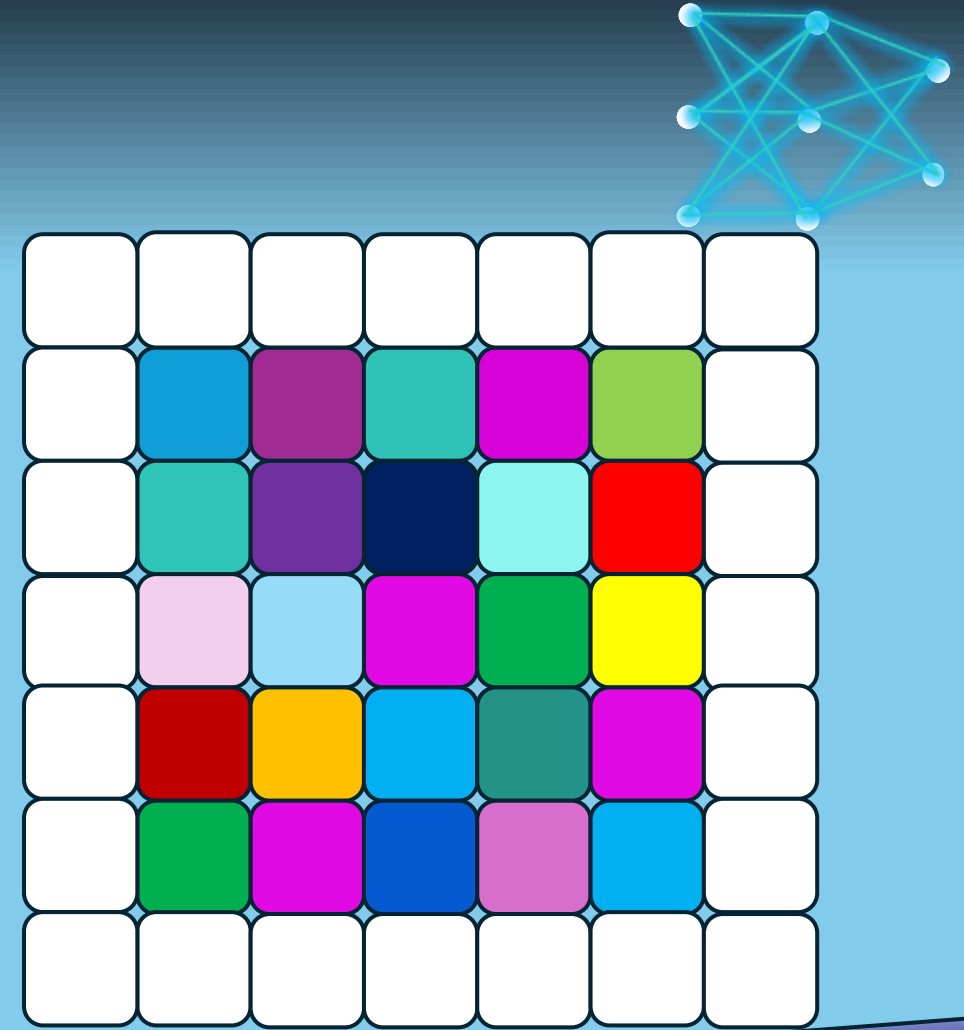
$$C(j, k) = \sum_l \sum_m I(j + l - 1, k + m - 1) K(l, m)$$

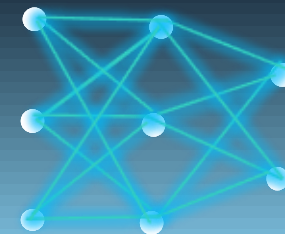
- Slide filter across image
- Compute local dot products
- Feature map = spatial correlation with learned kernel



Padding Solution

- Add extra pixels around image border
- Padding size: P
- **Output dimension:**
 $(J+2P-M+1) \times (K+2P-M+1)$





Types of Convolution

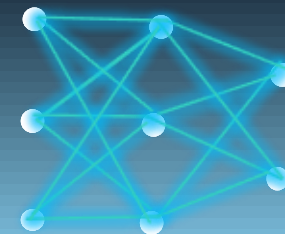
- **Valid Convolution**

- ✓ No padding: $P = 0$
- ✓ Output shrinks

- **Same Convolution**

$$P = \frac{M - 1}{2}$$

- ✓ The image and the feature map have the **same dimension**
- ✓ **Same convolution** preserves spatial dimensions



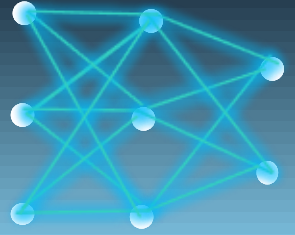
Effect on Feature Map Dimension

- For input dimension $J \times K$ and with the same stride horizontally and vertically, the number of elements in the feature map will be

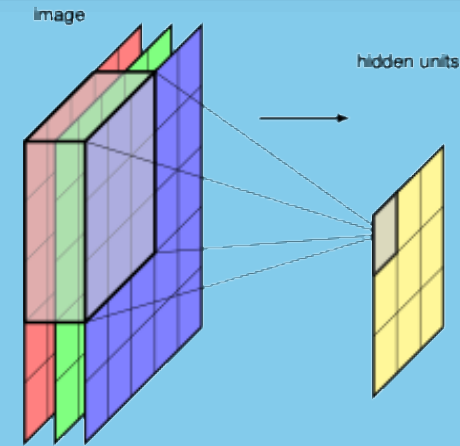
$$\left\lfloor \frac{J+2P-M}{S} \right\rfloor + 1 \times \left\lfloor \frac{K+2P-M}{S} \right\rfloor + 1$$

- For large images and small filter dim, the image map will be roughly a factor of $1/S$ smaller than the original image

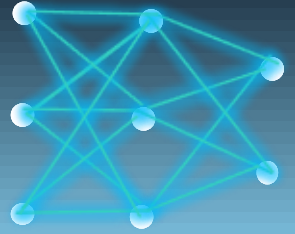
Filters for Multi-Channel Images



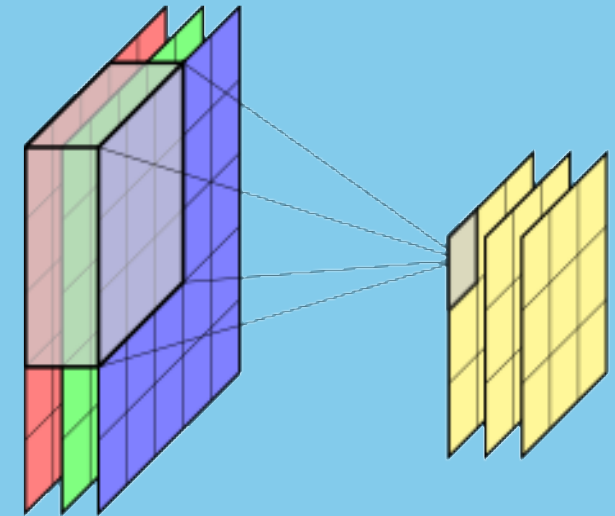
- A filter must span **all input channels**
- **Filter shape:** $M \times M \times C$
One $M \times M$ kernel per channel
- Outputs a **single feature map**
- **Output size (stride 1, no padding):** $(J - M + 1) \times (K - M + 1)$
- Each filter combines spatial + cross-channel information



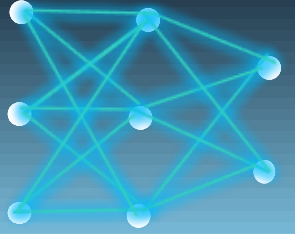
Multiple Filters and Feature Maps



- Use multiple filters: each with independent parameters
- Each filter produces its own feature map
- Multiple filters = richer feature representation

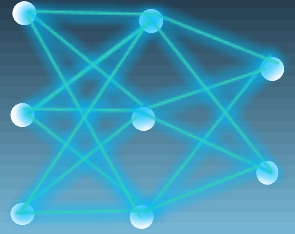


1×1 Convolutions

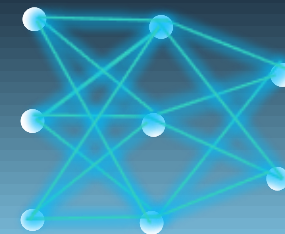


- Combines **channel information only**
- No spatial aggregation
- **Parameters:** C weights per filter + bias

Complementarity with Strided Convolutions



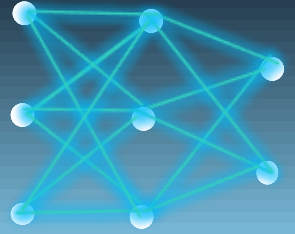
- Strided convolution reduces **spatial resolution**
- 1×1 convolution reduces or mixes **channel dimension**
- **Together** Control both spatial dimension and feature depth



Pooling – Why Is It Needed?

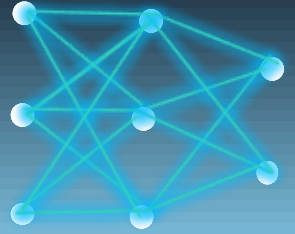
- Convolutional layers are **translation equivariant**
 - shift in input $\rightarrow$ shift in feature map
 - Valuable for applications such as finding the location of an object within an image
- **For other applications such as image classification, we want the output to be invariant to translations of the input**

Role of Pooling



- Pooling operates on feature maps Like convolution:
 - uses receptive fields
 - has stride and window size
- **Key difference: No learnable parameters**

Max Pooling



- **Each unit outputs the max function applied to the input values**
(maximum value in receptive field)

- **Intuition:**

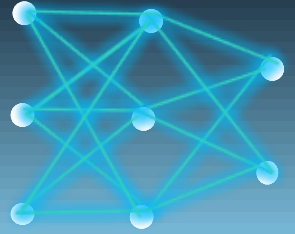
- Preserves strongest feature response
- Discards fine spatial details
- Feature exists somewhere in region

3	5	4	6
1	1	9	4
7	10	9	5
12	2	9	4

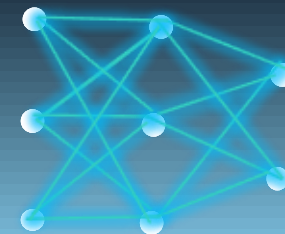


5	9
12	9

Pooling as Down-sampling

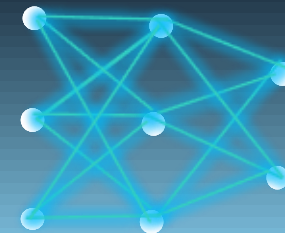


- As well as building in some **local translation invariance**, Pooling can also be used to **reduce the dimensionality** of the representation by down-sampling the feature map



What Information is Preserved?

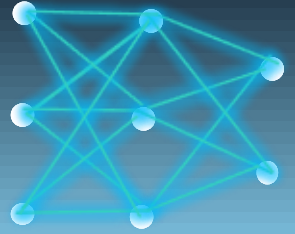
- Max-pooling preserves information on
 - presence of feature
 - strength of activation
- Pooling discards:
 - exact location within region
- **Interpretation:**
 - Feature maps = detectors
 - pooling = summarizer



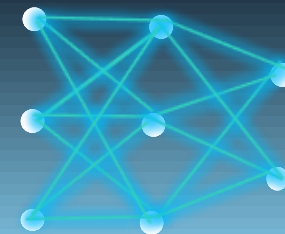
Other Pooling Functions

- **Max pooling:** strongest activation
- **Average pooling:** mean activation in region
- Both introduce **local translation invariance**
- Different ways of summarizing spatial evidence

Pooling Across Channels



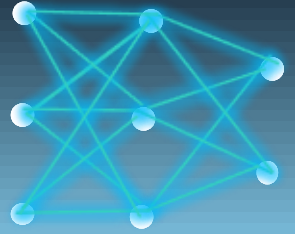
- Typically applied **per channel independently**
- **But can also** pool across channels
- **What for?**
- **Effect: Gives the network the potential to learn other invariances beyond simple translation invariance (e.g., rotation invariance)**
- enables more abstract invariances beyond translation



Pooling and Variable Image Sizes

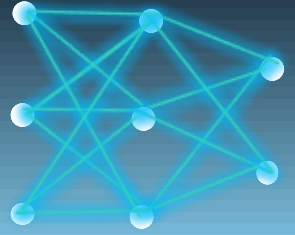
- Pooling allows a convolutional network to process **images of varying sizes**
- The **output** and some of the intermediate layers of CNN must have a **fixed size**
- **Solution:** Varying the stride length of the pooling according to the size of the image such that the number of pooled outputs remains constant
- **Result:** fixed-dimensional representation regardless of input size

Multilayer Convolutional Networks



- A single convolutional layer is analogous to a single **layer in a standard fully connected neural network**
- To model complex structure, we stack multiple layers
- **Key idea:** Depth enables hierarchical feature learning

Layer Parameterization

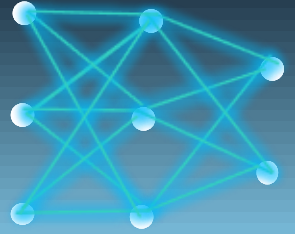


- Each convolutional layer is described by a filter tensor of dimensionality:

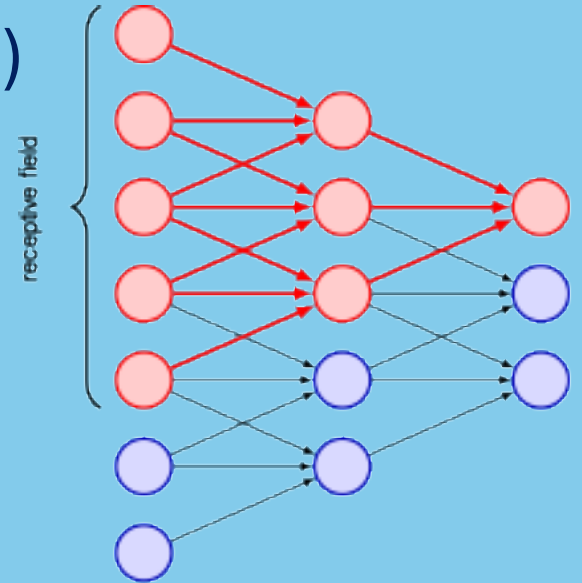
$$M \times M \times C_{in} \times C_{out}$$

- **Number of weight and bias parameters:** $(M^2 C_{in} + 1) C_{out}$
- Each such convolutional layer can optionally be followed by a pooling layer
- We can apply multiple such layers of convolution and pooling in succession. Output channels of layer l become input channels of layer $l + 1$

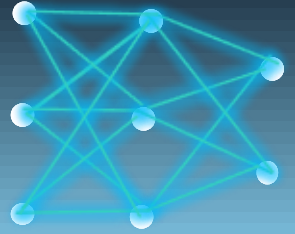
Locality and Receptive Fields



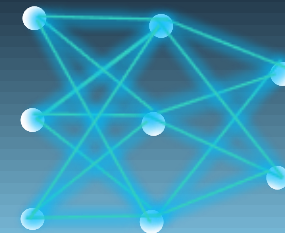
- Each neuron sees only a **local patch** (receptive field)
- **Key consequence in deep networks:**
 - receptive field grows with depth
 - later layers integrate broader context
 - Deep CNNs progressively expand spatial awareness



From Local Features to Global Decisions



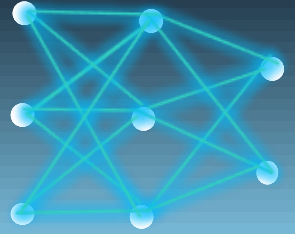
- Many tasks require **global image understanding**
 - e.g., classification
- **Challenge:** convolution is inherently local
- **Solution:** add one or two standard **fully connected layers** at the final stages of the network
- Aggregate global context for final prediction



Parameter Distribution in CNNs

- Convolutional layers:
 - **weight sharing** → few independent parameters
- Fully connected layers:
 - **dense connectivity** → many independent parameters
- **Key insight:**
 - FC layers can dominate model capacity
- Most “degrees of freedom” often concentrated at the top

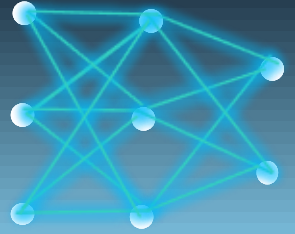
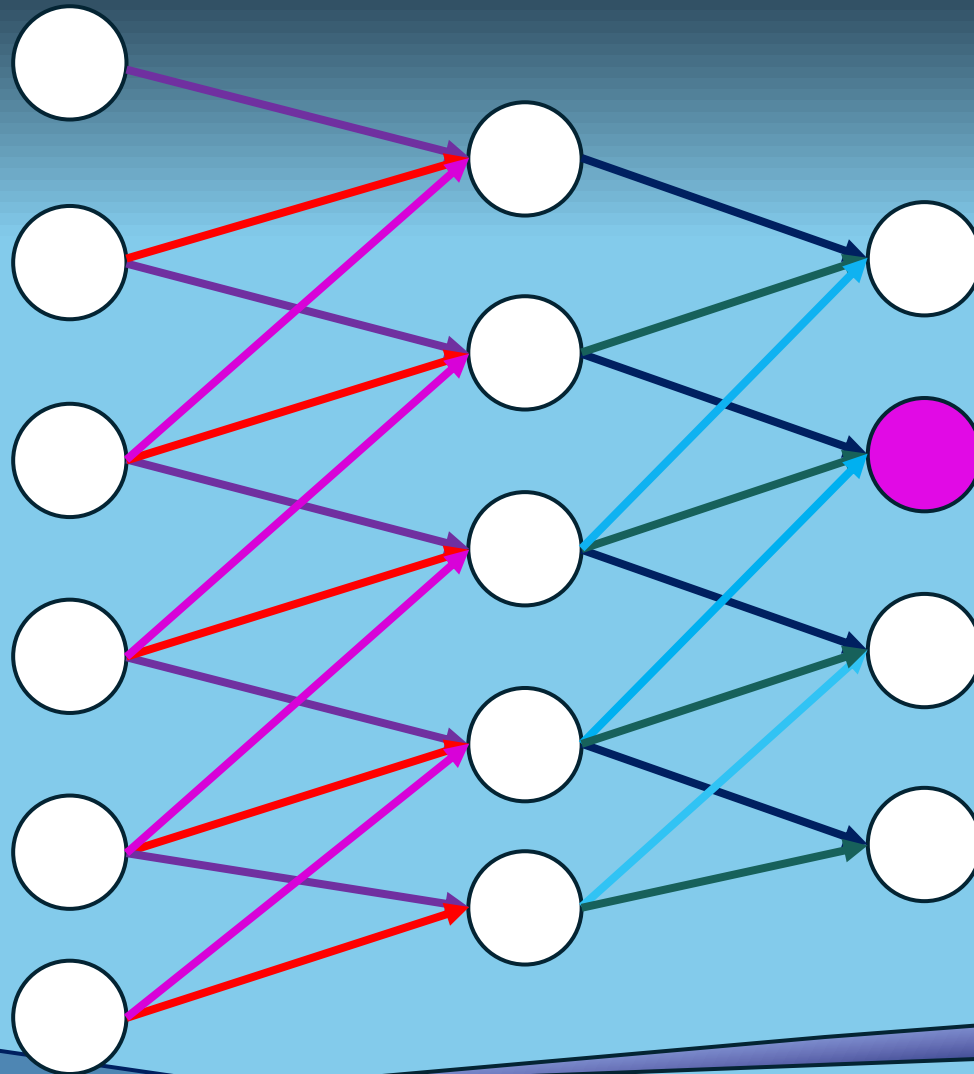
Complete CNN Architecture



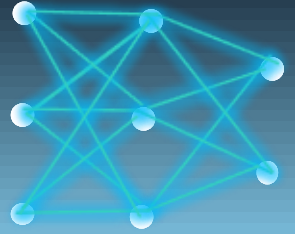
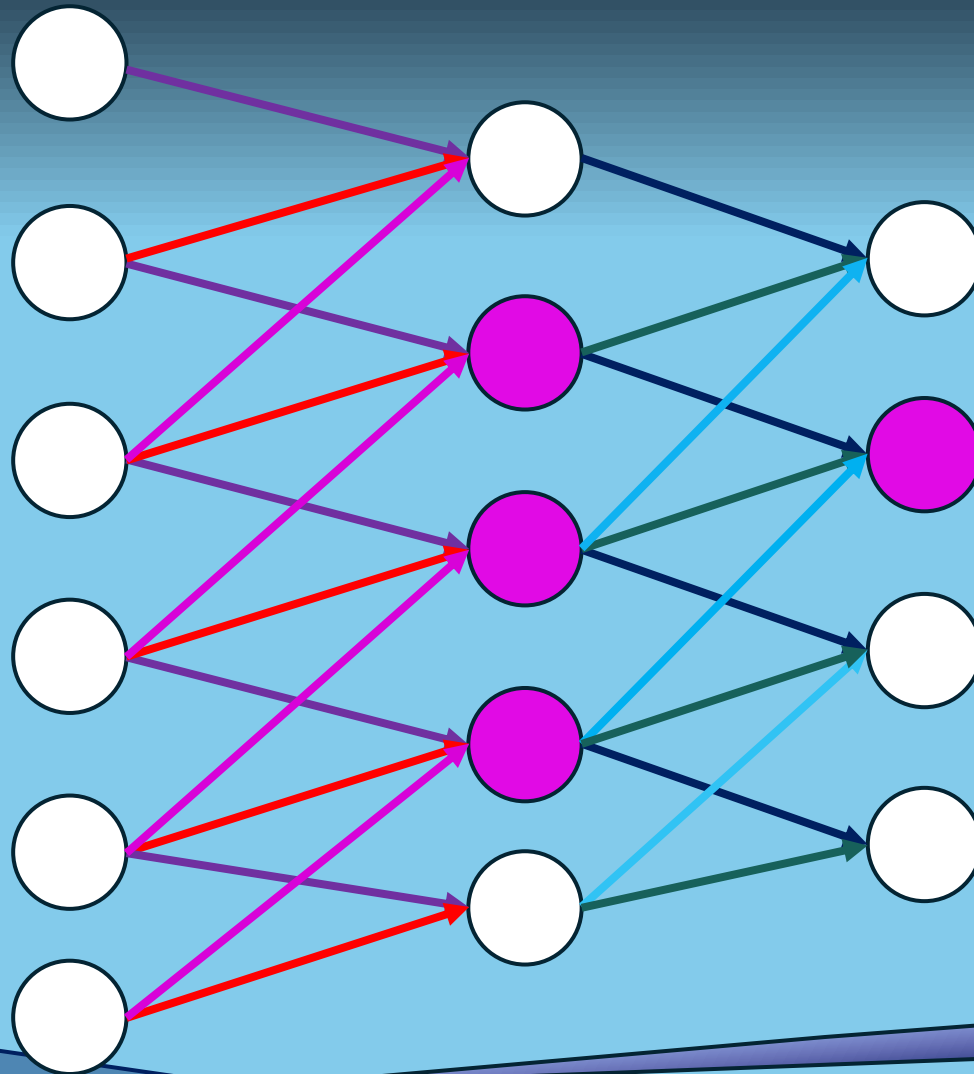
A full CNN typically includes:

Multiple layers of **convolutions** interspersed with **pooling** operations, and often with conventional **fully connected** layers in the final stages of the network

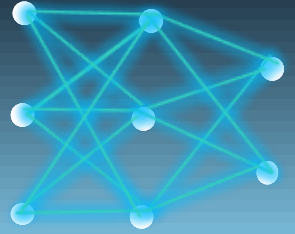
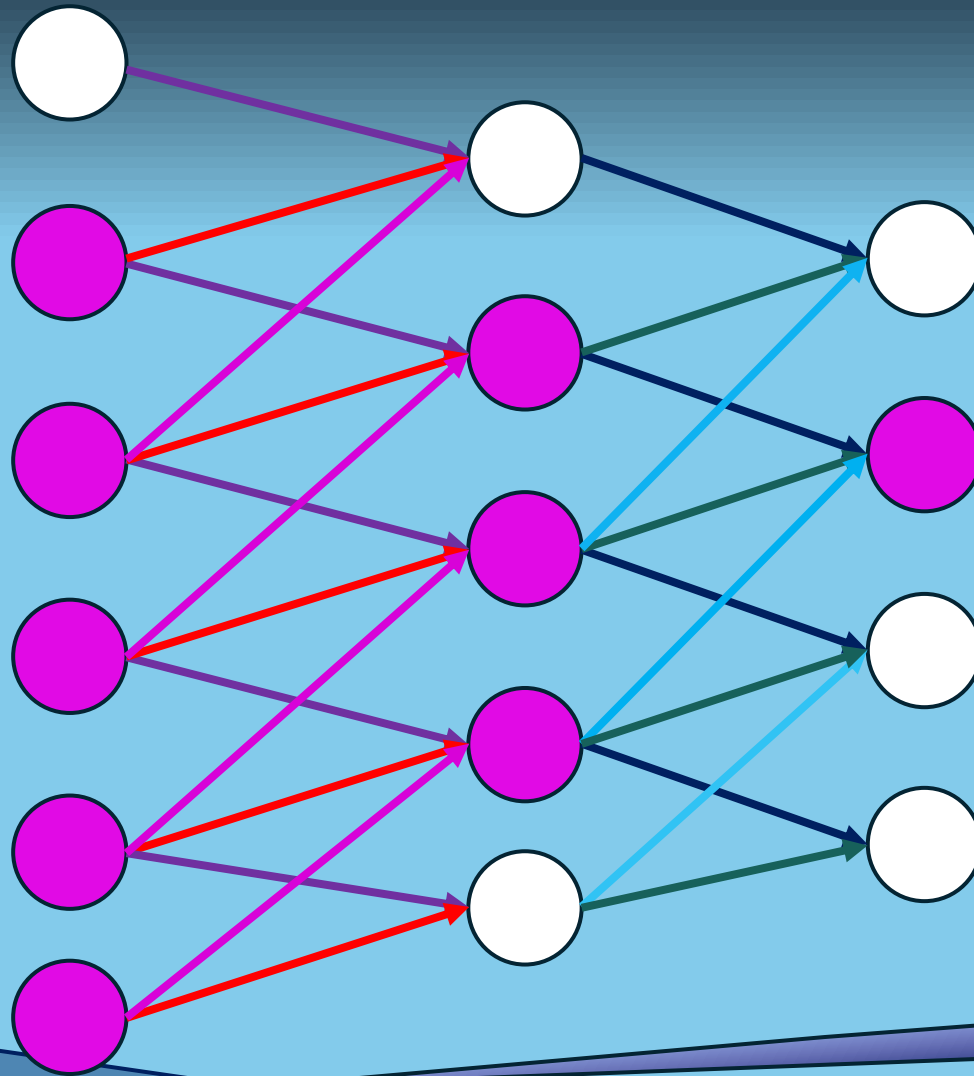
Receptive Field



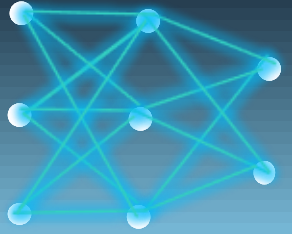
Receptive Field



Receptive Field



Receptive Field



r_l - the receptive field size of the final output feature map with respect to feature map at layer l

M_l kernel size (positive integer)

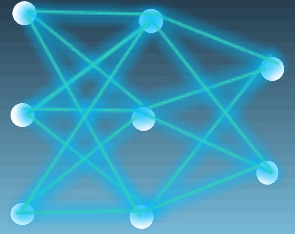
s_l stride (positive integer)

$$r_L = 1$$

$$r_{l-1} = s_l \cdot (r_l - 1) + 1 + (M_l - 1)$$

$$r_{l-1} = s_l \cdot (r_l - 1) + M_l$$

Receptive Field



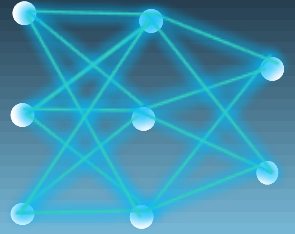
$$r_{l-1} = s_l \cdot (r_l - 1) + M_l$$

$$r_L = 1$$

$$r_{L-1} = s_L \cdot (r_L - 1) + M_L$$

$$r_{L-1} = M_L = (M_L - 1) + 1$$

Receptive Field



$$r_{l-1} = s_l \cdot (r_l - 1) + M_l$$

$$r_L = 1$$

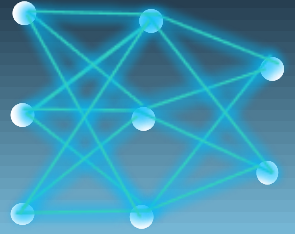
$$r_{L-1} = M_L = (M_L - 1) + 1$$

$$r_{L-2} = s_{L-1} \cdot (r_{L-1} - 1) + M_{L-1}$$

$$r_{L-2} = s_{L-1} \cdot (M_L - 1) + M_{L-1}$$

$$r_{L-2} = s_{L-1} \cdot (M_L - 1) + (M_{L-1} - 1) + 1$$

Receptive Field



$$r_{l-1} = s_l \cdot (r_l - 1) + M_l$$

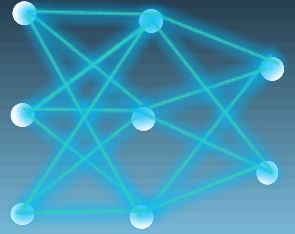
$$r_L = 1$$

$$r_{L-1} = M_L = (M_L - 1) + 1$$

$$r_{L-2} = s_{L-1} \cdot (M_L - 1) + (M_{L-1} - 1) + 1$$

$$r_{L-3} = s_{L-2} \cdot (r_{L-2} - 1) + M_{L-2} = s_{L-2} \cdot (s_{L-1} \cdot (M_L - 1) + (M_{L-1} - 1)) + M_{L-2}$$

Receptive Field



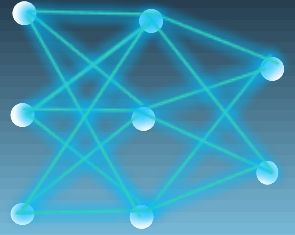
$$r_L = 1$$

$$r_{l-1} = s_l \cdot (r_l - 1) + M_l$$

$$r_{L-3} = s_{L-2} \cdot (r_{L-2} - 1) + M_{L-2} = s_{L-2} \cdot (s_{L-1} \cdot (M_L - 1) + (M_{L-1} - 1)) + M_{L-2}$$

$$r_{L-3} = s_{L-2} \cdot s_{L-1} \cdot (M_L - 1) + s_{L-2} \cdot (M_{L-1} - 1) + (M_{L-2} - 1) + 1$$

Receptive Field



$$r_L = 1$$

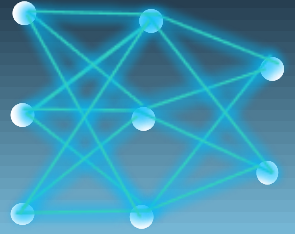
$$r_{L-1} = M_L = (M_L - 1) + 1$$

$$r_{L-2} = s_{L-1} \cdot (M_L - 1) + (M_{L-1} - 1) + 1$$

$$r_{L-3} = s_{L-2} \cdot s_{L-1} \cdot (M_L - 1) + s_{L-2} \cdot (M_{L-1} - 1) + (M_{L-2} - 1) + 1$$

$$r_0 = \sum_{l=1}^L \left((M_l - 1) \prod_{j=1}^{l-1} s_j \right) + 1$$

Receptive Field



$$r_0 = \sum_{l=1}^L \left((M_l - 1) \prod_{j=1}^{l-1} s_j \right) + 1$$

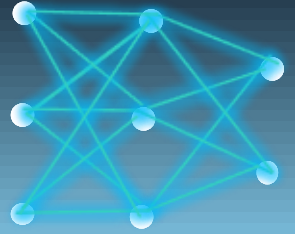
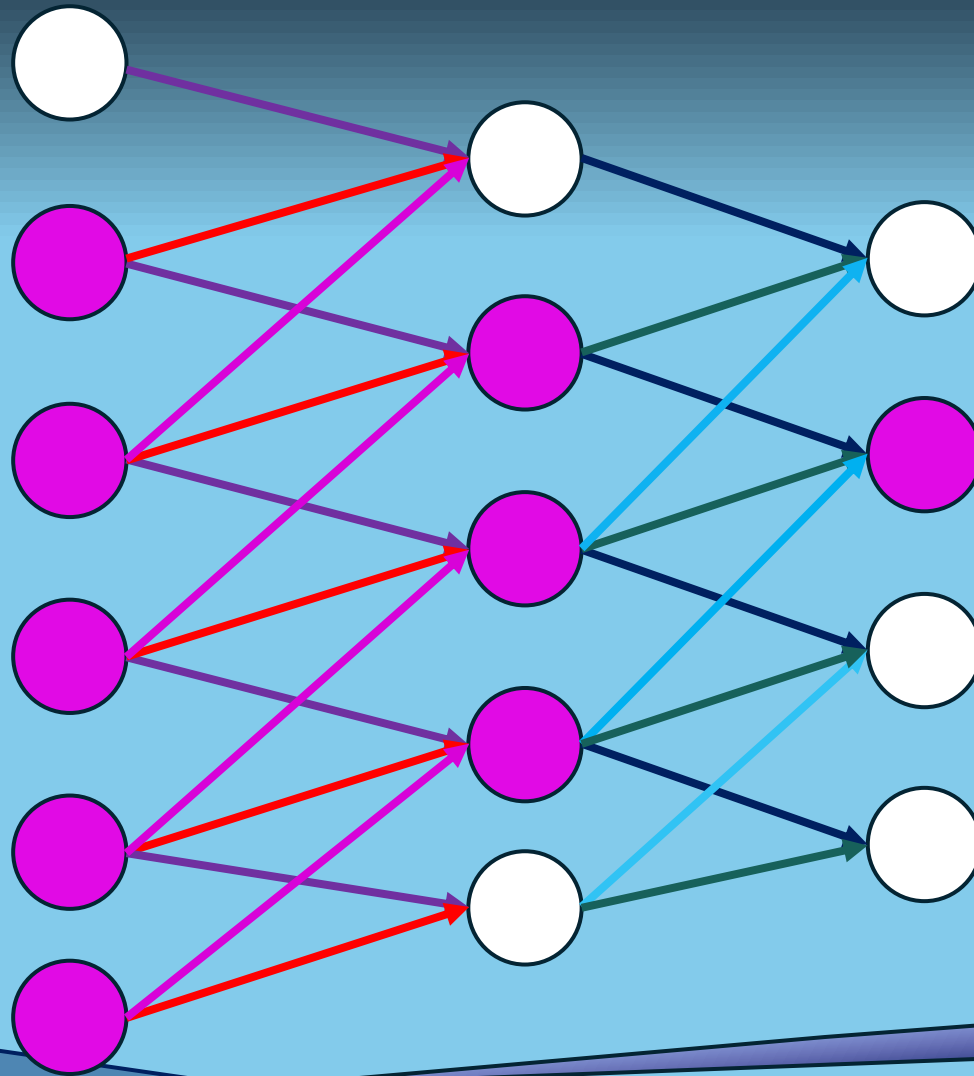
$$M_l = M \quad s_l = s$$

$$r_0 = (M - 1) \sum_{l=1}^L s^{l-1} + 1$$

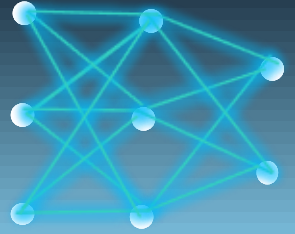
$$r_0 = (M - 1) \frac{s^L - 1}{s - 1} + 1$$

Receptive Field

$$r_0 = (M - 1) \frac{s^L - 1}{s - 1} + 1$$



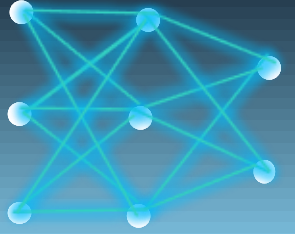
Receptive Field - Proof



$$r_{l-1} = s_l \cdot (r_l - 1) + M_l$$

$$r_{l-1} \prod_{i=1}^{l-1} s_i = s_l \cdot (r_l - 1) \prod_{i=1}^{l-1} s_i + M_l \prod_{i=1}^{l-1} s_i$$
$$r_{l-1} \prod_{i=1}^{l-1} s_i = r_l \cdot \prod_{i=1}^l s_i + M_l \prod_{i=1}^{l-1} s_i - \prod_{i=1}^l s_i$$

Receptive Field - Proof

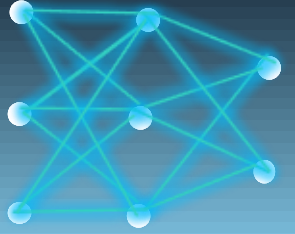


$$r_{l-1} \prod_{i=1}^{l-1} s_i = r_l \cdot \prod_{i=1}^l s_i + M_l \prod_{i=1}^{l-1} s_i - \prod_{i=1}^l s_i$$

$$A_{l-1} = A_l + M_l \prod_{i=1}^{l-1} s_i - \prod_{i=1}^l s_i$$

$$A_l - A_{l-1} = \prod_{i=1}^l s_i - M_l \prod_{i=1}^{l-1} s_i$$

Receptive Field - Proof

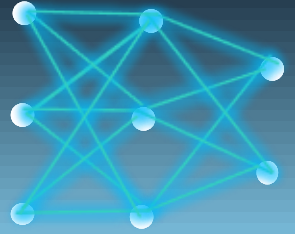


$$A_l - A_{l-1} = \prod_{i=1}^l s_i - M_l \prod_{i=1}^{l-1} s_i$$

$$\sum_{l=1}^L A_l - A_{l-1} = \sum_{l=1}^L \prod_{i=1}^l s_i - M_l \prod_{i=1}^{l-1} s_i$$

$$A_L - A_0 = \prod_{i=1}^L s_i - r_0 = \sum_{l=1}^L \prod_{i=1}^l s_i - M_l \prod_{i=1}^{l-1} s_i$$

Receptive Field - Proof

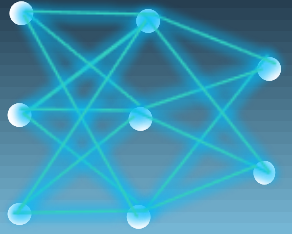


$$A_L - A_0 = \prod_{i=1}^L s_i - r_0 = \sum_{l=1}^L \prod_{i=1}^l s_i - M_l \prod_{i=1}^{l-1} s_i$$

$$r_0 = \prod_{i=1}^L s_i - \sum_{l=1}^L \prod_{i=1}^l s_i - M_l \prod_{i=1}^{l-1} s_i = \sum_{l=1}^L M_l \prod_{i=1}^{l-1} s_i - \sum_{l=1}^{L-1} \prod_{i=1}^l s_i$$

$$r_0 = \sum_{l=1}^L M_l \prod_{i=1}^{l-1} s_i - \sum_{l=1}^L \prod_{i=1}^{l-1} s_i + 1 \quad r_0 = \sum_{l=1}^L \left((M_l - 1) \prod_{i=1}^{l-1} s_i \right) + 1$$

Examples

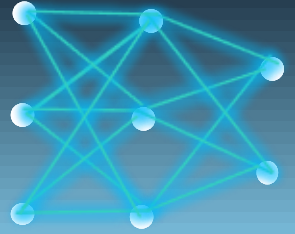


$$r_0 = (M - 1) \frac{s^L - 1}{s - 1} + 1$$

$$L = 2, M = 3, s = 2$$

$$L = 2, M = 3, s = 4$$

Design Choices (Hyperparameters)



- CNN design involves multiple choices:
- number of layers
- number of channels per layer
- filter sizes
- stride widths
- pooling strategy
- **Challenge:** A wide variety of different architectures have been explored, although in practice it is difficult to make a systematic comparison of hyperparameter values using hold-out data due to the high computational cost of training each candidate configuration

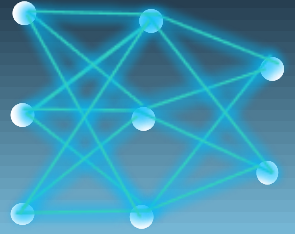
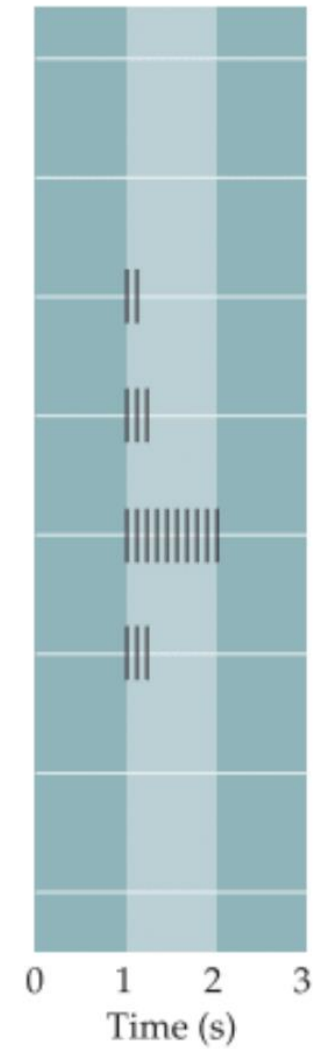
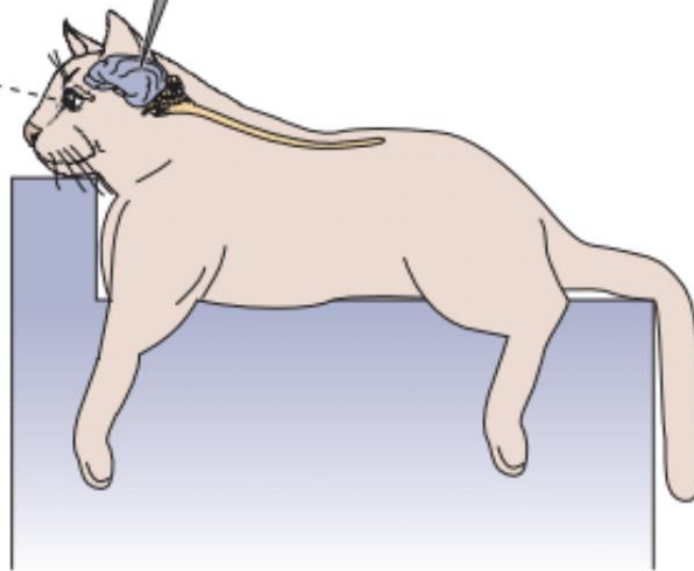
Hubel & Wiesel (1959)

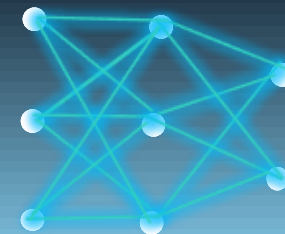
Light bar stimulus
projected on screen



Recording from visual cortex

Record





Visualizing Trained CNNs

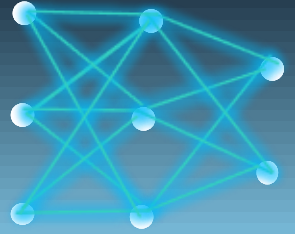
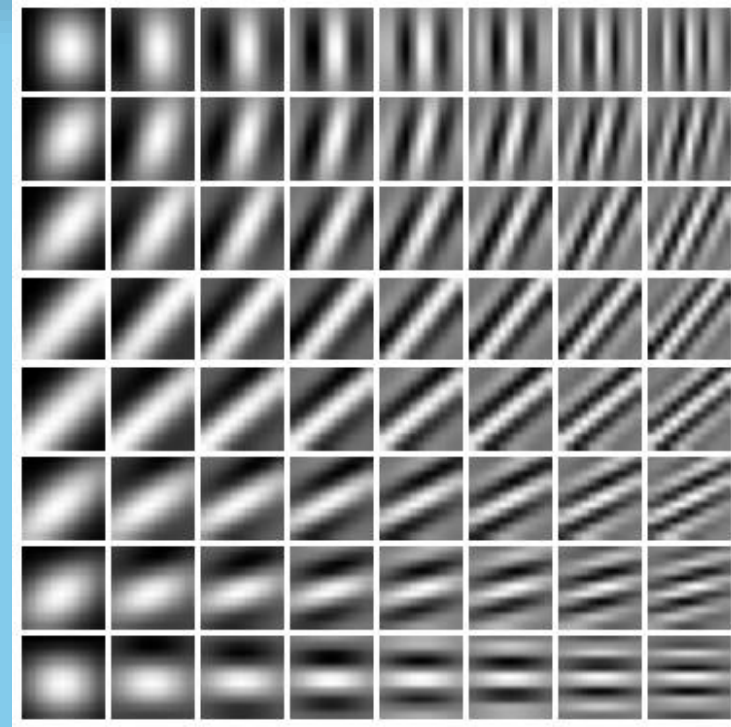
- In their pioneering work, Hubel and Wiesel (1959) measured the electrical responses of individual neurons in the visual cortex of cats while presenting visual stimuli to the cats' eyes
- They discovered that some neurons have a strong response to visual inputs with a **simple edge oriented** at a **particular angle** and **located at a particular position** within the visual field, whereas other stimuli generated relatively little response in those neurons

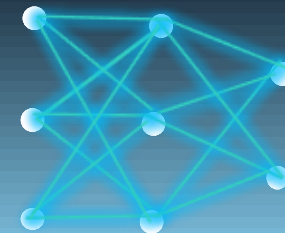
Gabor filters

$$G(x, y) = A \exp(-\alpha \tilde{x}^2 - \beta \tilde{y}^2) \sin(\omega \tilde{x} + \phi)$$

$$\tilde{x} = (x - x_0) \cos(\theta) + (y - y_0) \sin(\theta)$$

$$\tilde{y} = -(x - x_0) \sin(\theta) + (y - y_0) \cos(\theta)$$

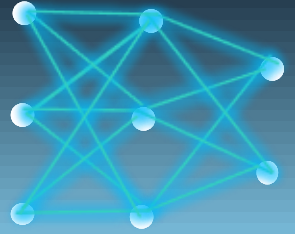




Visualizing Trained Filters

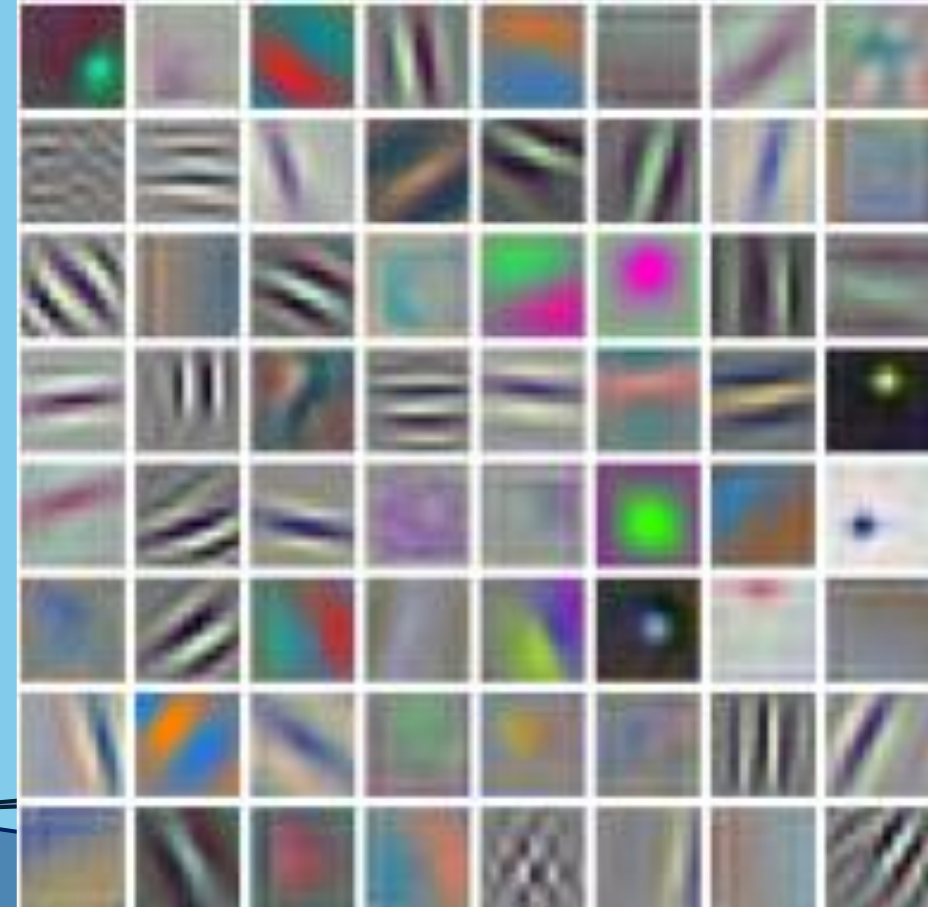
- Convolutional Neural Networks are highly interpretable in terms of the types of features they can learn
- Suppose we have a trained deep CNN and we wish to explore what the hidden units have learned to detect

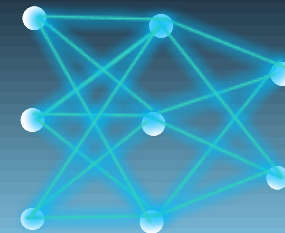
Visualizing Trained Filters



- For the filters in the first convolutional layer – this is relatively straightforward!

Examples of learned filters from the first layer of AlexNet





Visualizing Trained Filters

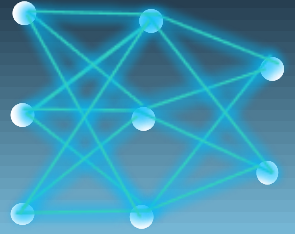
- The subsequent layers are harder to interpret
- One approach – present a large number of image patches to the network and see which produce the highest activation value in any particular hidden unit

Visualizing Trained Filters



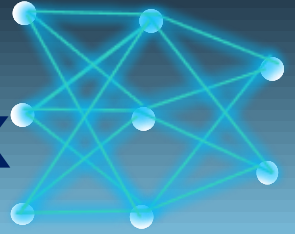
Examples of image patches (taken from a validation set) that produce the strongest activation in the hidden units in a network having 5 conv layers trained on imageNet data

Visualization



- Once the network has been trained in a supervised way, one can attempt to reconstruct the portions of the image that most activate a given feature
- The portions of an image that are most likely to activate a particular hidden feature or a class are identified
- This goal can be achieved with various types of backpropagation and optimization formulas

Visualizing the Features of a Trained Network



- Given an activation of a feature anywhere in the neural network for a particular input image, visualize the portions of the input to which that feature is responding the most
- Given a particular feature anywhere in the neural network, find a fantasy image that is likely to activate that feature the most
- In both these cases, the easiest approach to visualize the impact of specific features is to use gradient-based methods



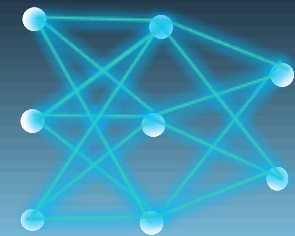
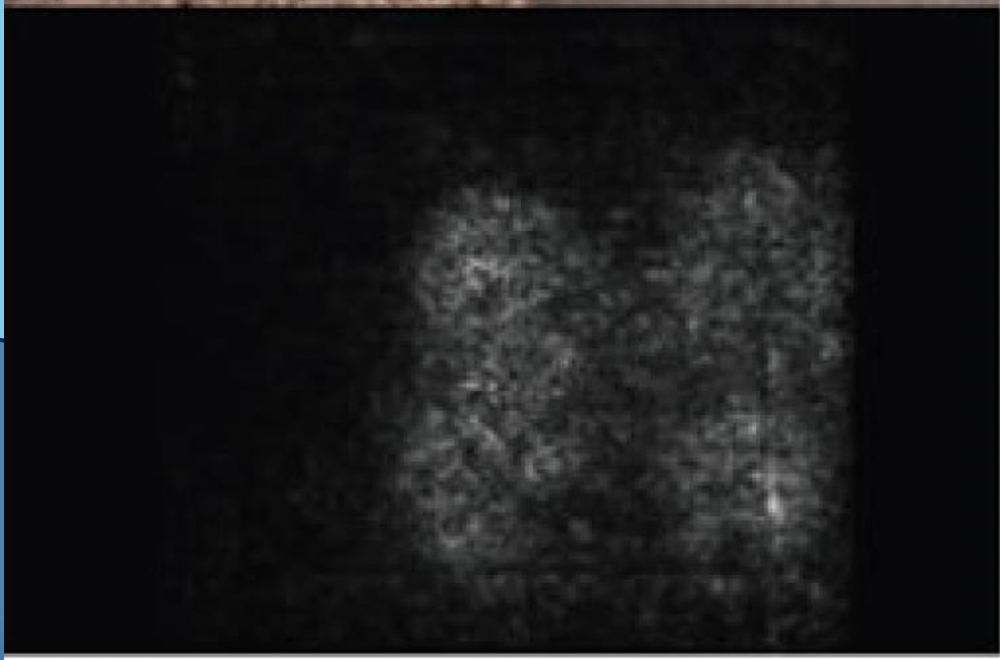
Gradient-based Visualization of Activated Features

- Our main goal is to determine the sensitivity of the hidden/ output features with respect to different pixels of the input image
- The softmax probability of “banana” will be relatively insensitive to small changes in those portions of the image that are irrelevant to the recognition of banana. The values of the gradient will be close to zero for such irrelevant regions, whereas the portions of the image that define a banana will have large magnitudes

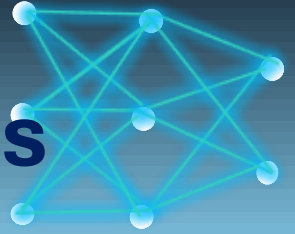


Gradient-based Visualization of Activated Features

- To visualize this volume, we first convert it to grayscale by taking the maximum of the absolute magnitude of the gradient over the three RGB channels
- The bright portions of this grayscale visualization will tell us which portion of the input image are relevant to the banana



Gradient-based Visualization of Activated Features

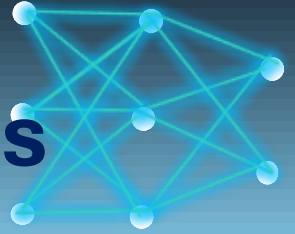


6th layer

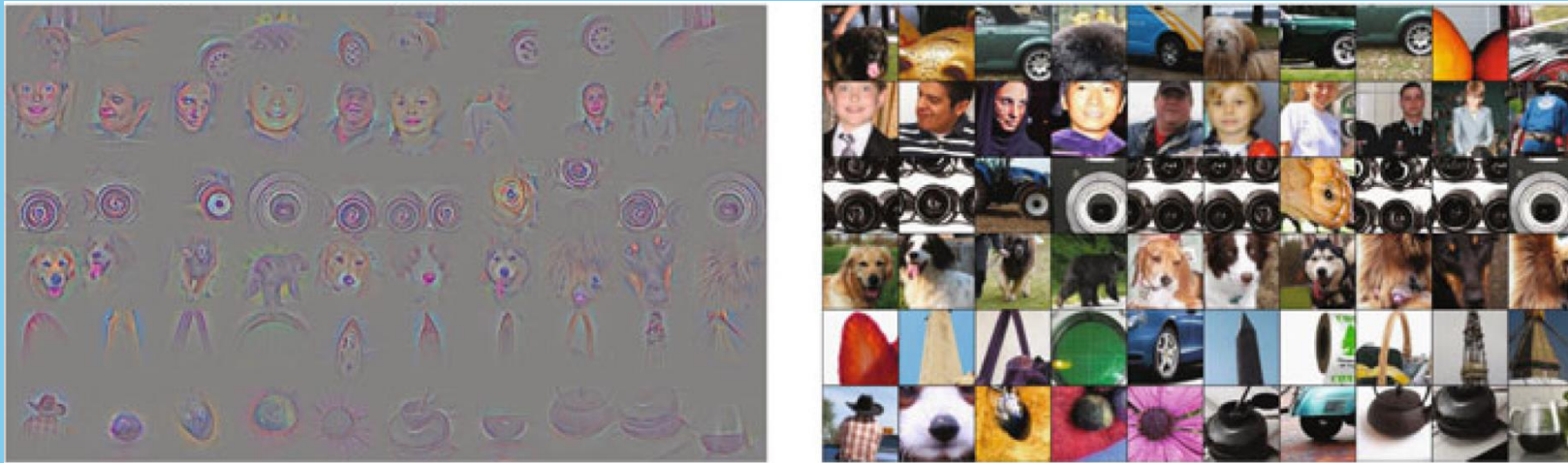


Examples of the image crops corresponding to specific images in which a particular neuron in a hidden layer is highly activated
Each row corresponds to a particular hidden feature, the variations in that row are caused by the different choices of image
The images that most activated the feature were selected

Gradient-based Visualization of Activated Features



9th layer

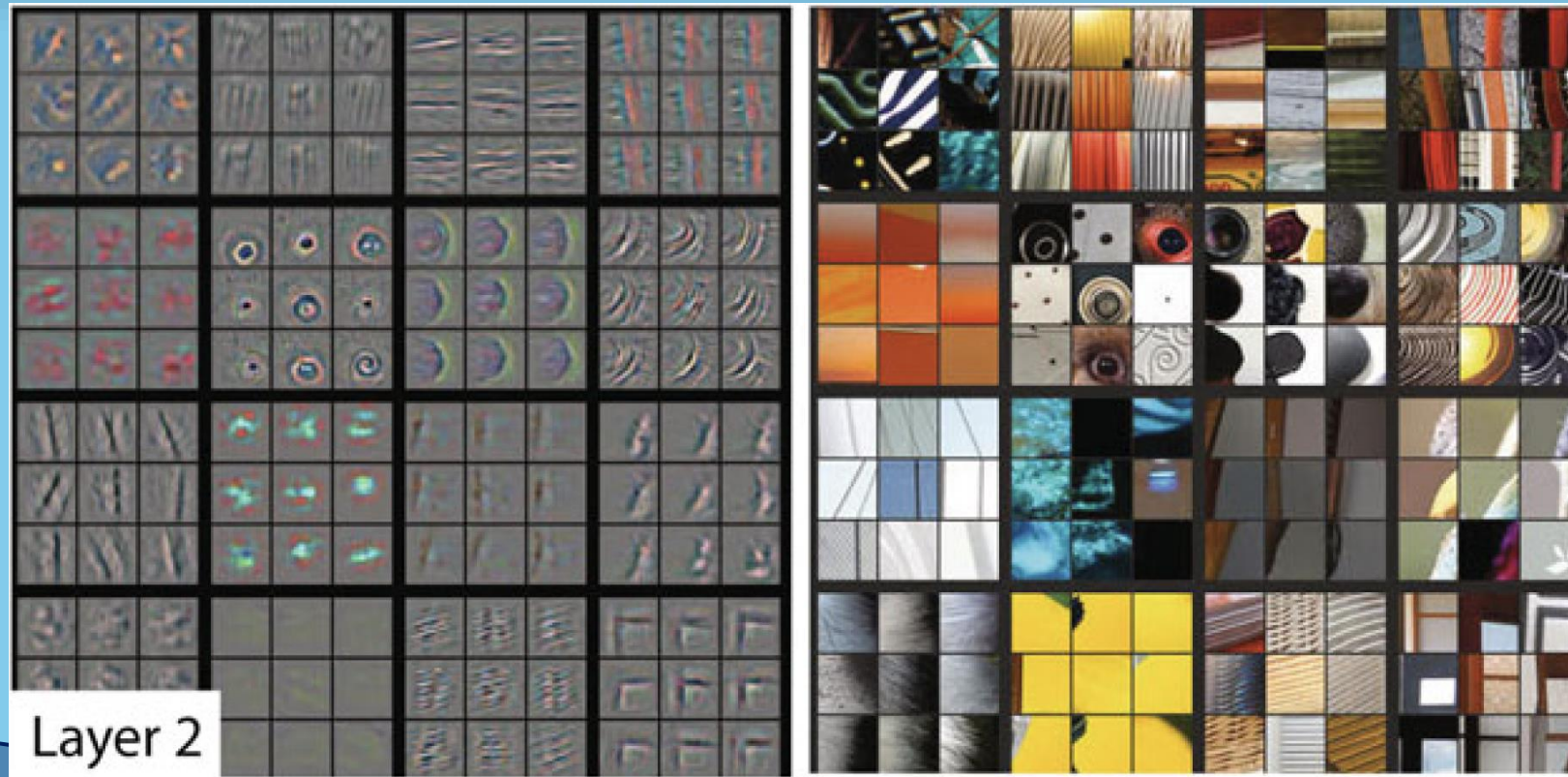


The images correspond to larger regions of the input image containing more complex shapes

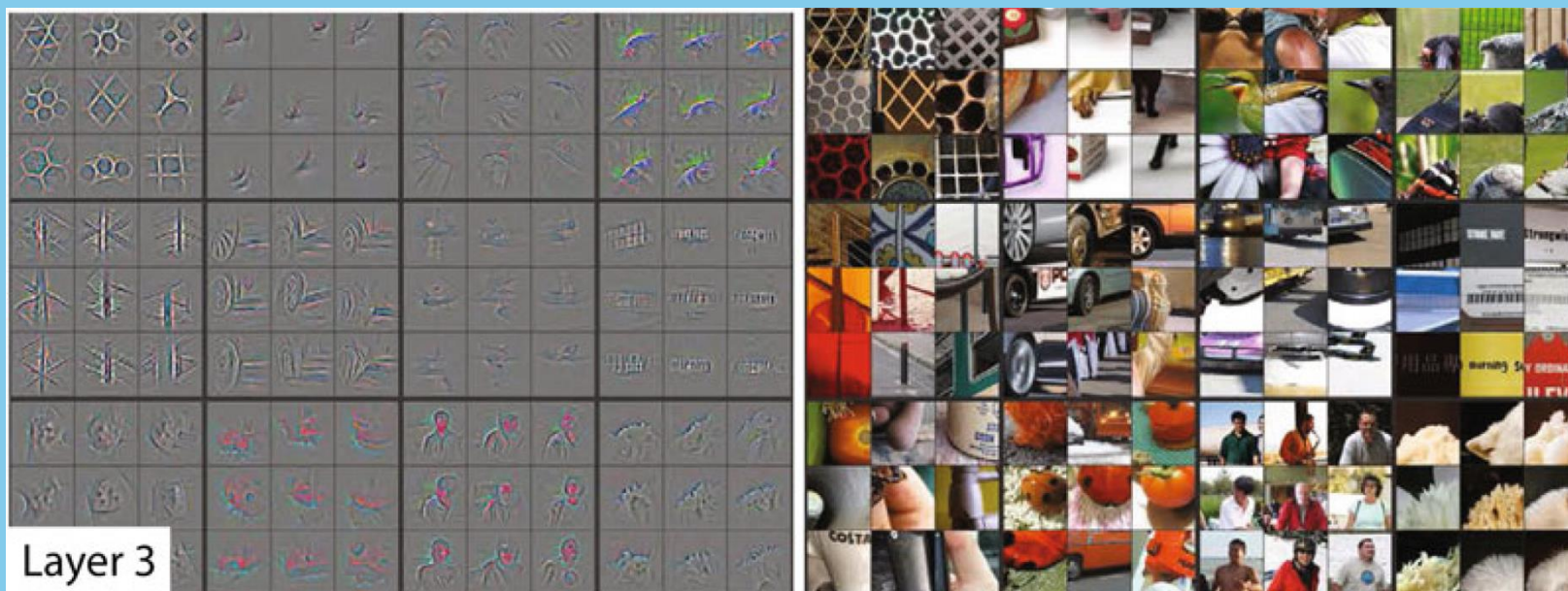
Top-9 Largest Activations in Each Feature Map

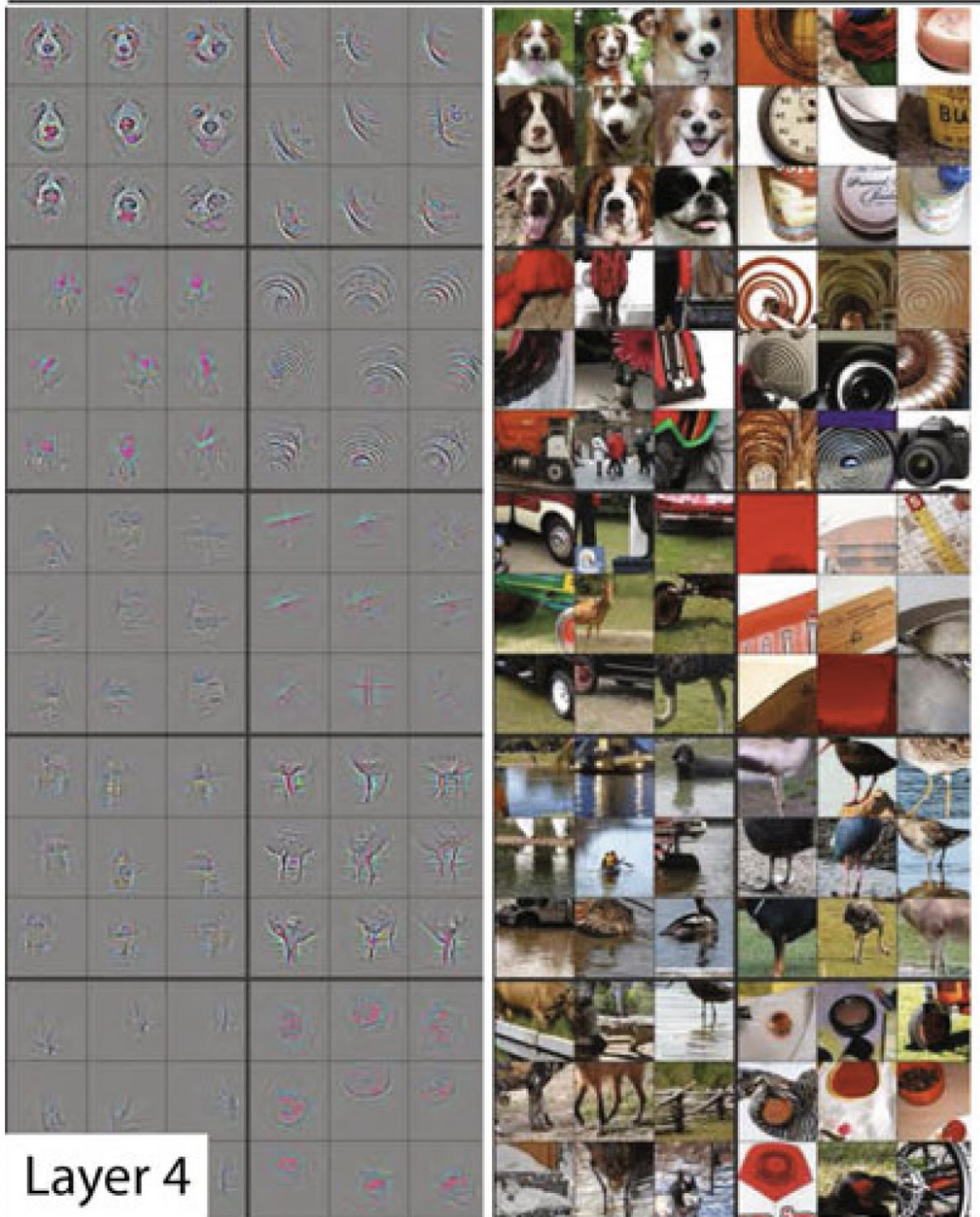


Top-9 Largest Activations in Each Feature Map

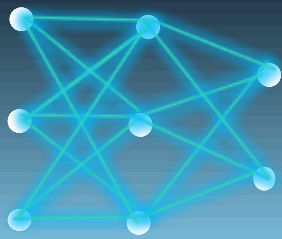


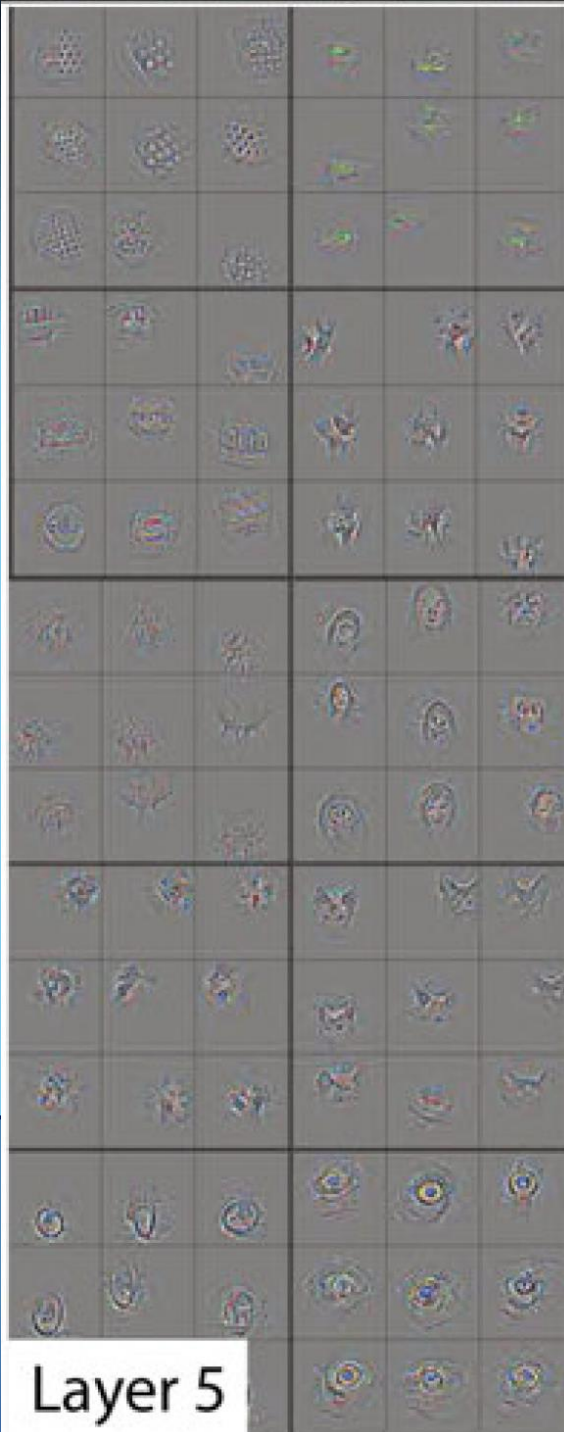
Top-9 Largest Activations in Each Feature Map



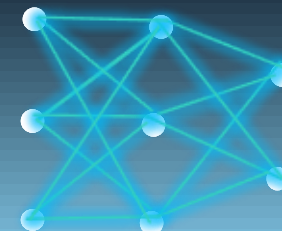
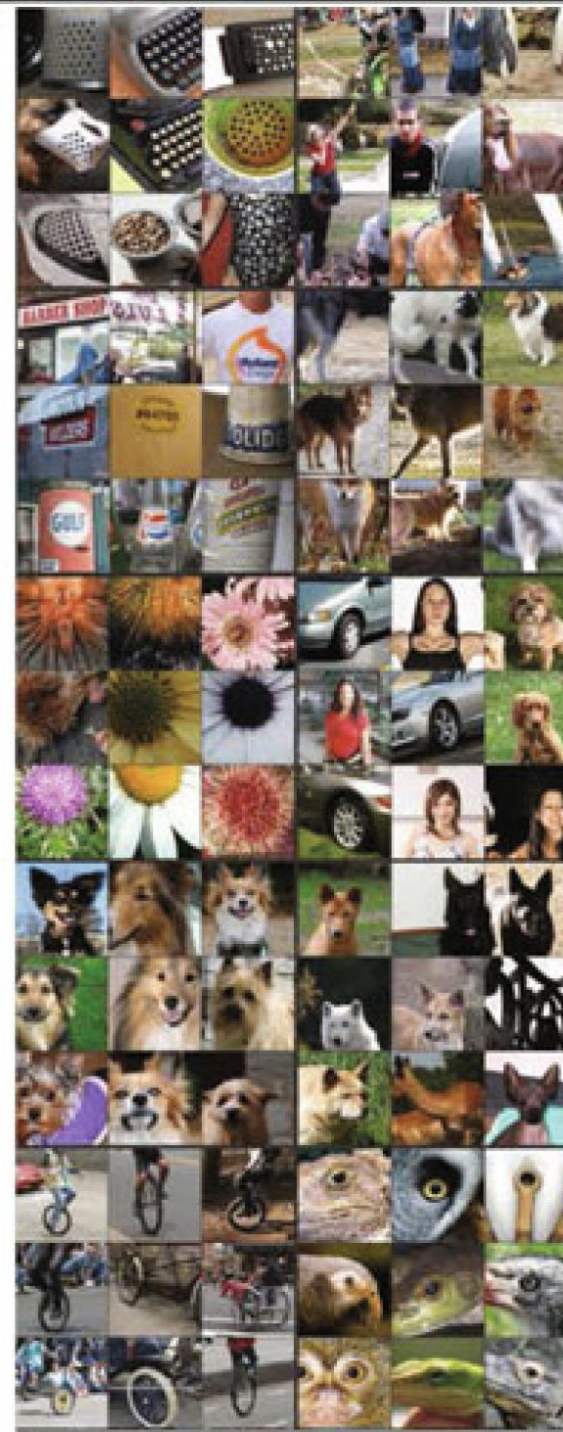


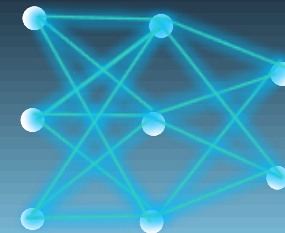
Layer 4





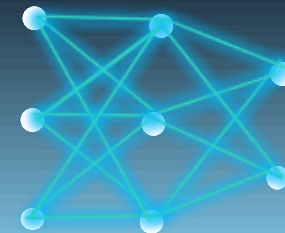
Layer 5





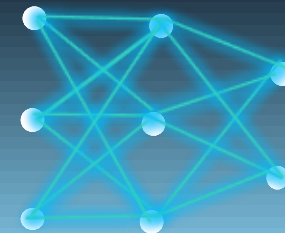
Visualization of Activated Features

- The features in early layers tend to be more generic, and they can be used across a wider variety of data sets
- The features in later layers tend to be more specific to individual data sets
- This is a key property exploited in **transfer learning applications**



Visualizing Trained Filters

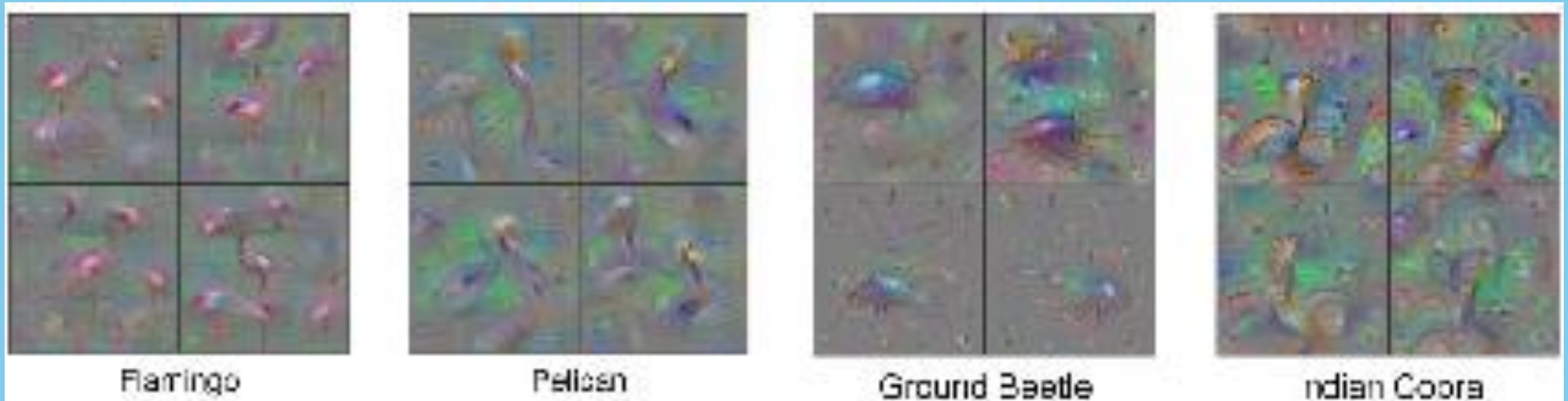
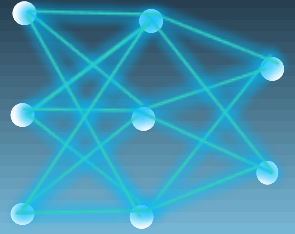
- We can extend this technique to go beyond simply selecting image patches from the validation set and instead perform a numerical optimization over the input variables to maximize the activation of a particular unit
- Because the output units generally have a softmax activation function, it is better to maximize the pre-activation value that feeds into the softmax rather than the class probability directly, as this ensures the optimization depends on only one class



Visualizing Trained Filters

- Unconstrained optimization of the output-unit activation leads to individual pixel values being driven to infinity and also creates high-frequency structure that is difficult to interpret, and so some form of regularization is required to find solutions that are closer to natural images
- Yosinski used a regularization function comprising the sum of squares of the pixel values along with a procedure that alternates gradient-based updates to the image pixel values with a blurring operation to remove high-frequency structure and a clipping operation that sets to zero those pixels values that make only small contributions to the class label

Visualizing Trained Filters



Synthetic Images generated by maximizing the class probability with respect to the image pixel channel values for a trained convolutional classifier [Yosinski]

Synthesized Images that Activate a Feature



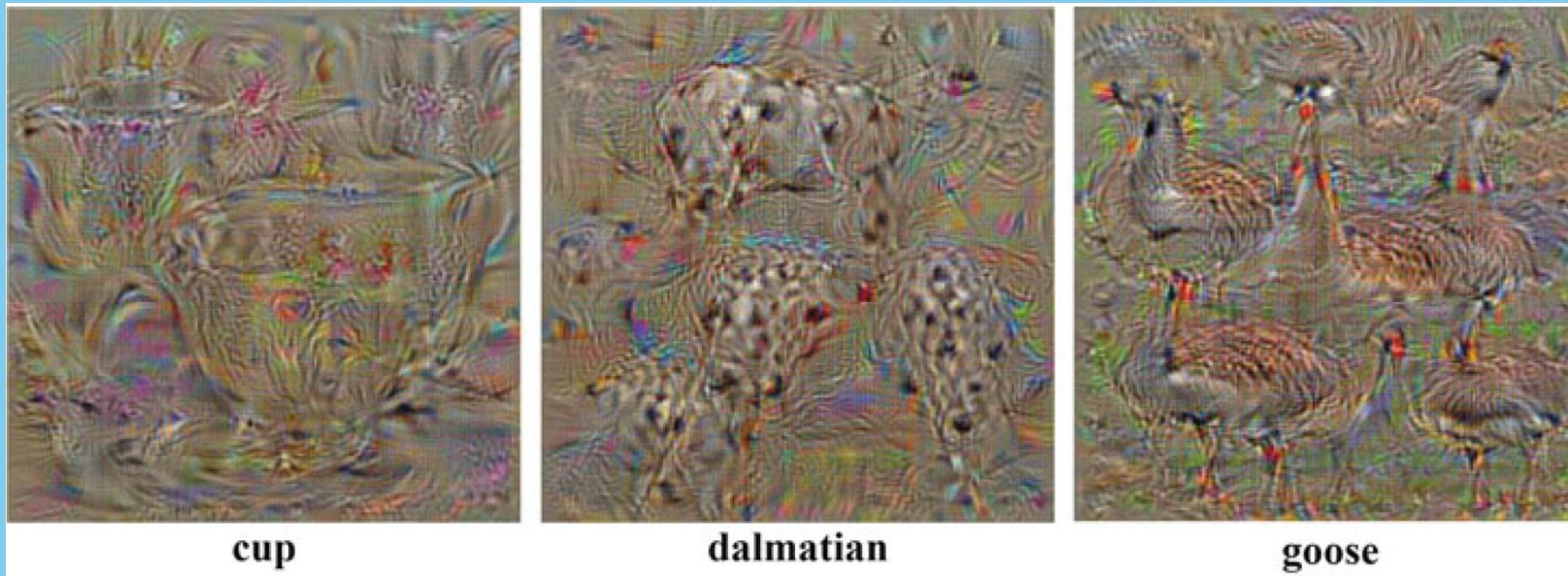
- What kind of image patch would maximally activate a particular neuron?

$$\text{Maximize}_x J(\mathbf{x})$$

$$J(\mathbf{x}) = O - \lambda \|\mathbf{x}\|^2$$

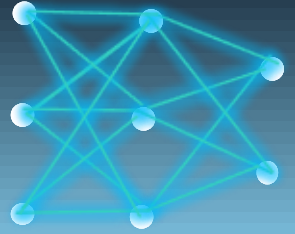
$$\mathbf{x}^{(t)} = \mathbf{x}^{(t-1)} + \alpha \nabla_{\mathbf{x}} J(\mathbf{x}^{(t-1)})$$

Synthesized Images that Activate a Feature



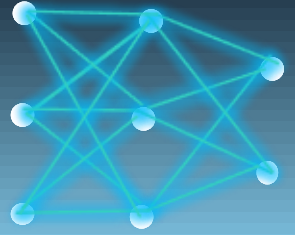
Examples of synthesized images with respect to particular class labels

ImageNet



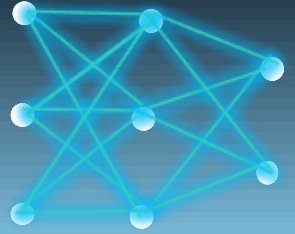
- Over 14 million labeled high-resolution images
- More than 22,000 categories

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)



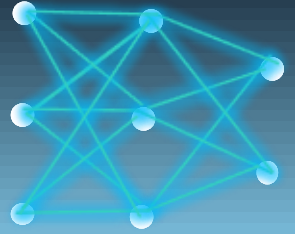
- 1.2 Million training Images
- 50,000 validation images
- 150,000 test images
- A subset of ImageNet with roughly 1,000 images in each of 1,000 categories
- Classes range from everyday objects (dogs, cats, chairs, cars) to fine-grained categories (specific bird species, dog breeds,...)

ILSVRC



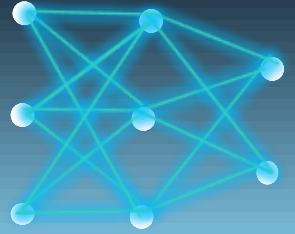
- On ImageNet it is customary to report two error rates:
top-1 and top-5
- Top-5 error rate is the **fraction of test images** for which the **correct label is not among the five labels considered most probable** by the model

AlexNet



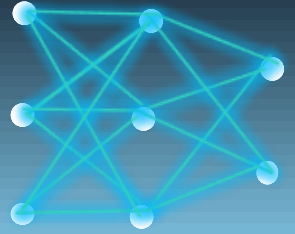
- The Winner of the 2012 ILSVRC competition
- 60 million parameters and 650,000 neurons
- **5 convolutional layers**, some of which are followed by max-pooling layers, and **3 fully-connected layers** with a final 1,000-way softmax
- Non-saturating neurons and a very efficient GPU implementation
- Dropout

AlexNet

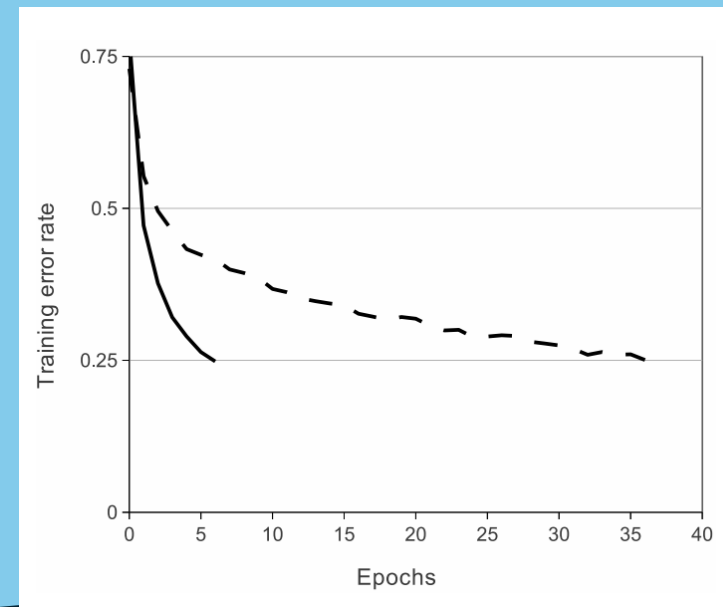


- Down-sampled the images to a fixed resolution of 256x256

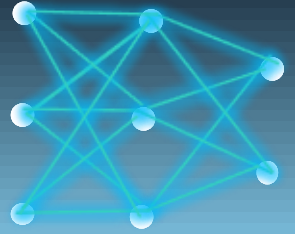
AlexNet



- Deep convolutional neural networks with ReLUs train several times faster than their equivalents with tanh units
- Faster learning has a great influence on the performance of large models trained on large datasets

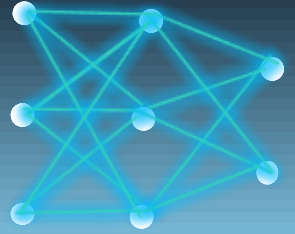


AlexNet – Training on Multiple GPUs



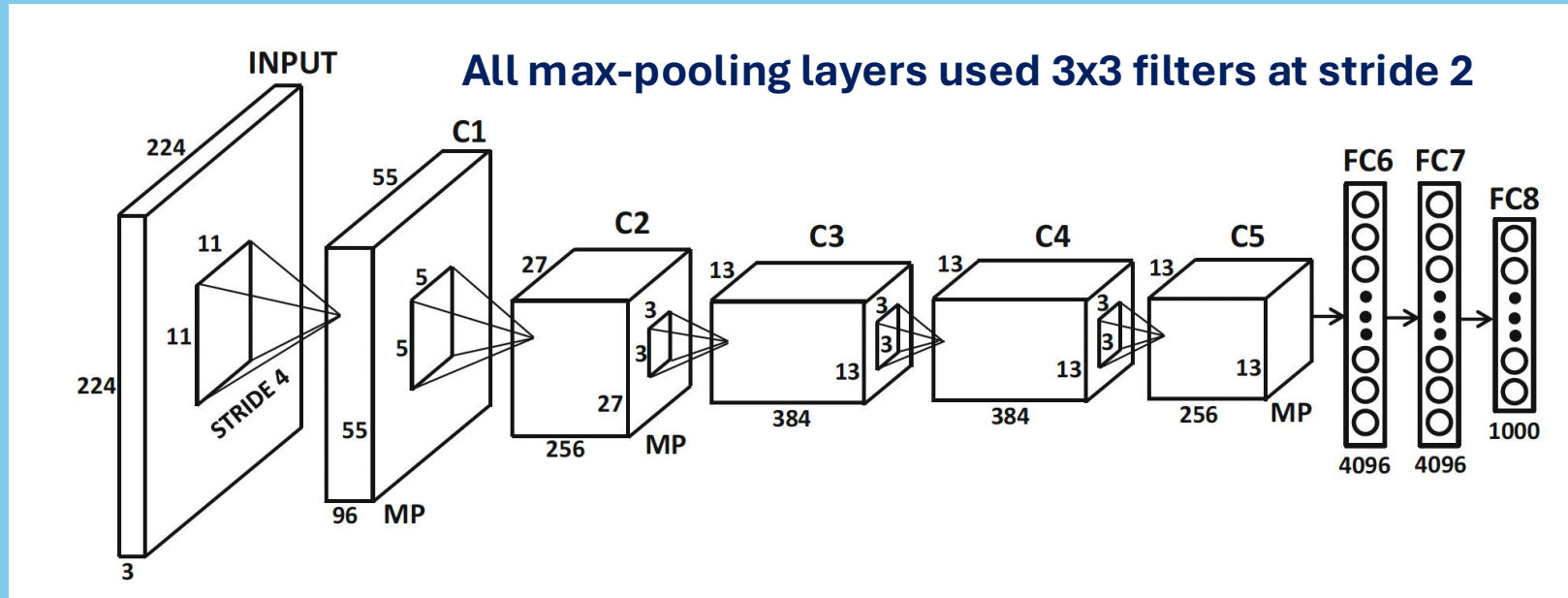
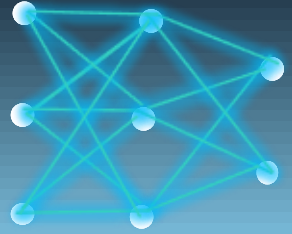
- A single GPU limits the maximum size of the networks that can be trained on it
- The network was spread across two GPUs
- The scheme puts half of the kernels on each GPU
- This scheme reduces their top-1 and top-5 error rates by 1.7% and 1.2%, respectively, as compared to a net with half as many kernels in each convolution layer trained on one GPU

Overlapping Pooling



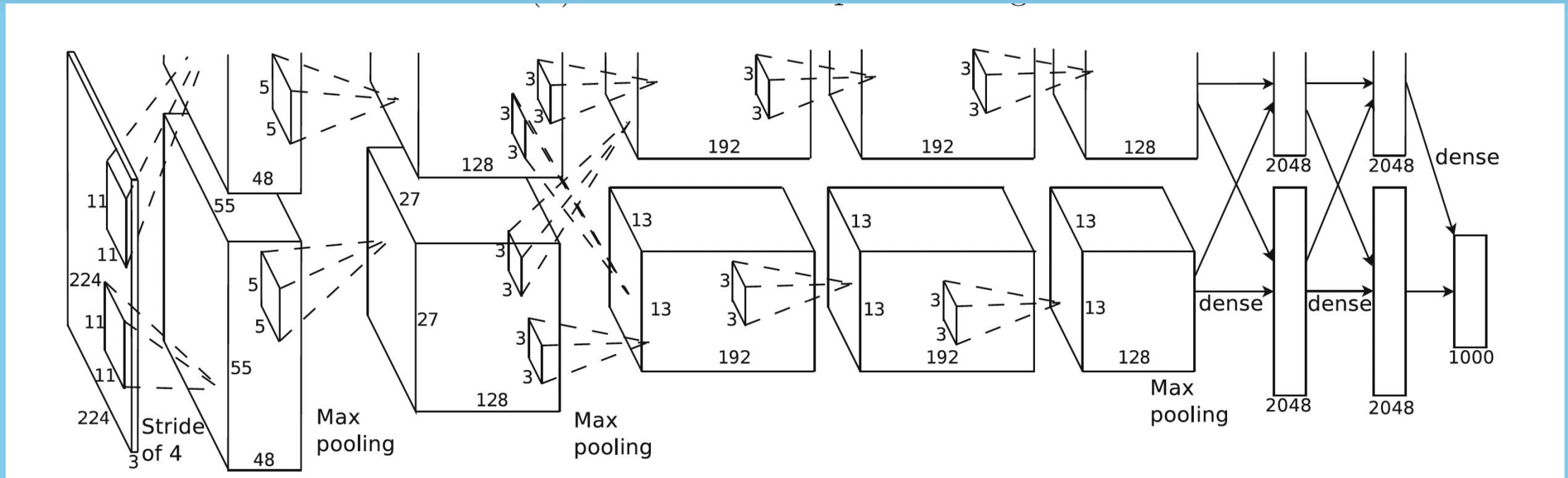
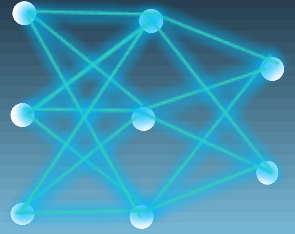
- Traditionally, the neighborhoods summarized by adjacent pooling units do not overlap
- If we set $s < z$, we obtain overlapping pooling ($s = 2, z = 3$)
- This scheme reduces the top-1 and top-5 error rates by 0.4% and 0.3%, respectively, as compared with the non-overlapping scheme $s = 2, z = 2$, which produces output of equivalent dimensions

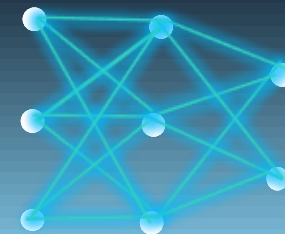
AlexNet - Overall Architecture



$$\left\lfloor \frac{J+2P-M}{S} \right\rfloor + 1 \times \left\lfloor \frac{K+2P-M}{S} \right\rfloor + 1$$

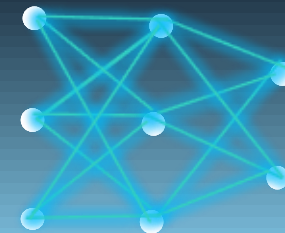
AlexNet





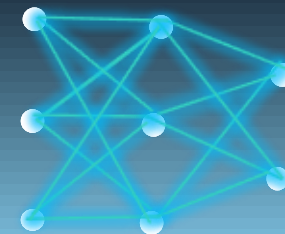
Reducing Overfit

- AlexNet has **60 million parameters**
- Two primary ways to **combat overfitting**:
 - **Data Augmentation**:
 - (Image **translations**, horizontal **reflections**) X2048
 - **Altering the intensities** of the RGB channels in training images – object identity is invariant to changes in the intensity and color of the illumination
 - **Dropout** in the first two fully connected layers ($p=0.5$)
 - dropout roughly **doubles the number of iterations** required to converge
 - Without dropout, the network exhibits substantial overfitting
- This scheme reduces the top-1 error rate by 1%



AlexNet – Parameter Choices

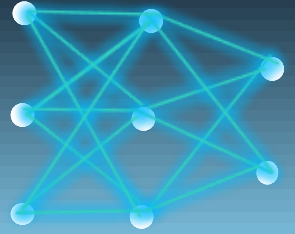
- L2 regularization was used with a parameter of 5×10^{-4}
- Dropout was used by sampling units at a probability of 0.5
- Momentum-based stochastic gradient descent was used for training with parameter value of 0.9
- The batch-size was 128



AlexNet – Parameter Choices

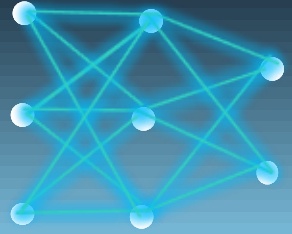
- The learning rate was 0.01, although it was eventually reduced a couple of times as the method began to converge (divided by 10 when the validation error stopped improving with the current learning rate)
- Initialization of weights in each layer - zero-mean Gaussian distribution with std 0.01
- The network was trained for roughly 90 cycles through the training set of 1.2 million images
- The training time of AlexNet was 5 to 6 days on two GPUs

AlexNet



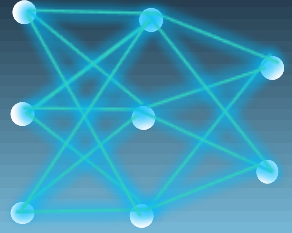
- The final top-5 error rate was about 15.4%
- The use of a single convolutional network provided a top-5 error-rate of 18.2%, although using an ensemble of 7 models provided a top-5 error-rate of 15.4%
- AlexNet is considered a fundamental advancement within the field of computer vision because of the large margin with which it won the ILSVRC contest. This success rekindled interest in deep learning in general and CNN in particular

ZFNet



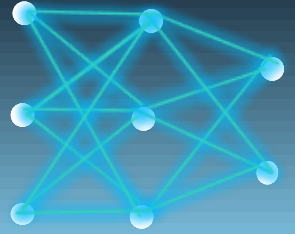
- A variant of ZFNet was the winner of the ILSVRC competition in 2013
- Its architecture was heavily based on AlexNet
- Most changes were associated with differences in hyperparameter choices

ZFNet



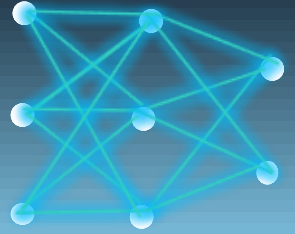
	<i>AlexNet</i>	<i>ZFNet</i>
Volume:	$224 \times 224 \times 3$	$224 \times 224 \times 3$
Operations:	Conv 11×11 (stride 4)	Conv 7×7 (stride 2), MP
Volume:	$55 \times 55 \times 96$	$55 \times 55 \times 96$
Operations:	Conv 5×5 , MP	Conv 5×5 (stride 2), MP
Volume:	$27 \times 27 \times 256$	$13 \times 13 \times 256$
Operations:	Conv 3×3 , MP	Conv 3×3
Volume:	$13 \times 13 \times 384$	$13 \times 13 \times 512$
Operations:	Conv 3×3	Conv 3×3
Volume:	$13 \times 13 \times 384$	$13 \times 13 \times 1024$
Operations:	Conv 3×3	Conv 3×3
Volume:	$13 \times 13 \times 256$	$13 \times 13 \times 512$
Operations:	MP, Fully connect	MP, Fully connect
FC6:	4096	4096
Operations:	Fully connect	Fully connect
FC7:	4096	4096
Operations:	Fully connect	Fully connect
FC8:	1000	1000
Operations:	Softmax	Softmax

ZFNet

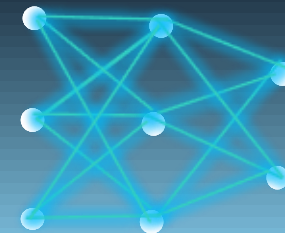


- This architecture reduced the top-5 error rate to 14.8% from 15.4%
- Further increases in width/depth reduced the error rate to 11.1%
- The architecture of ZFNet was made wider and deeper, and the results were submitted to ILSVRC in 2013 under the name Clarifai, which was a company founded by the first author
- This entry was the winning entry of the ILSVRC competition in 2013

VGG



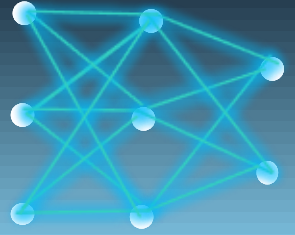
- VGG was a top performing entry on ILSVRC in 2014, but it was not the winner
- The best performing versions had 16 or more layers
- The winner was GoogleNet with top-5 error rate of 6.7% in comparison with the top-5 error rate of 7.3% for VGG
- VGG was important because it illustrated several important design principles that eventually became standard in future architectures



VGG

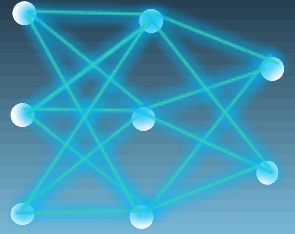
- Reduced filter sizes but increased depth
- Architectures with greater depth and reduced filter size require fewer parameters
- VGG always uses filters with spatial footprint 3x3 and pooling of size 2x2
- Conv with stride 1 and padding of 1
- Pooling at stride 2 (non-overlapping regions) – reduces the spatial footprint by a factor of 2

VGG



- The number of filters was often increased by a factor of 2 after each max-pooling
- A final max-pooling is performed just before creating the fully connected layer which reduces the spatial footprint further to 7
- The first fully connected layer has dense connections between 4096 neurons and a 7x7x512 volume
- Most of the parameters of the neural network are hidden in these connections

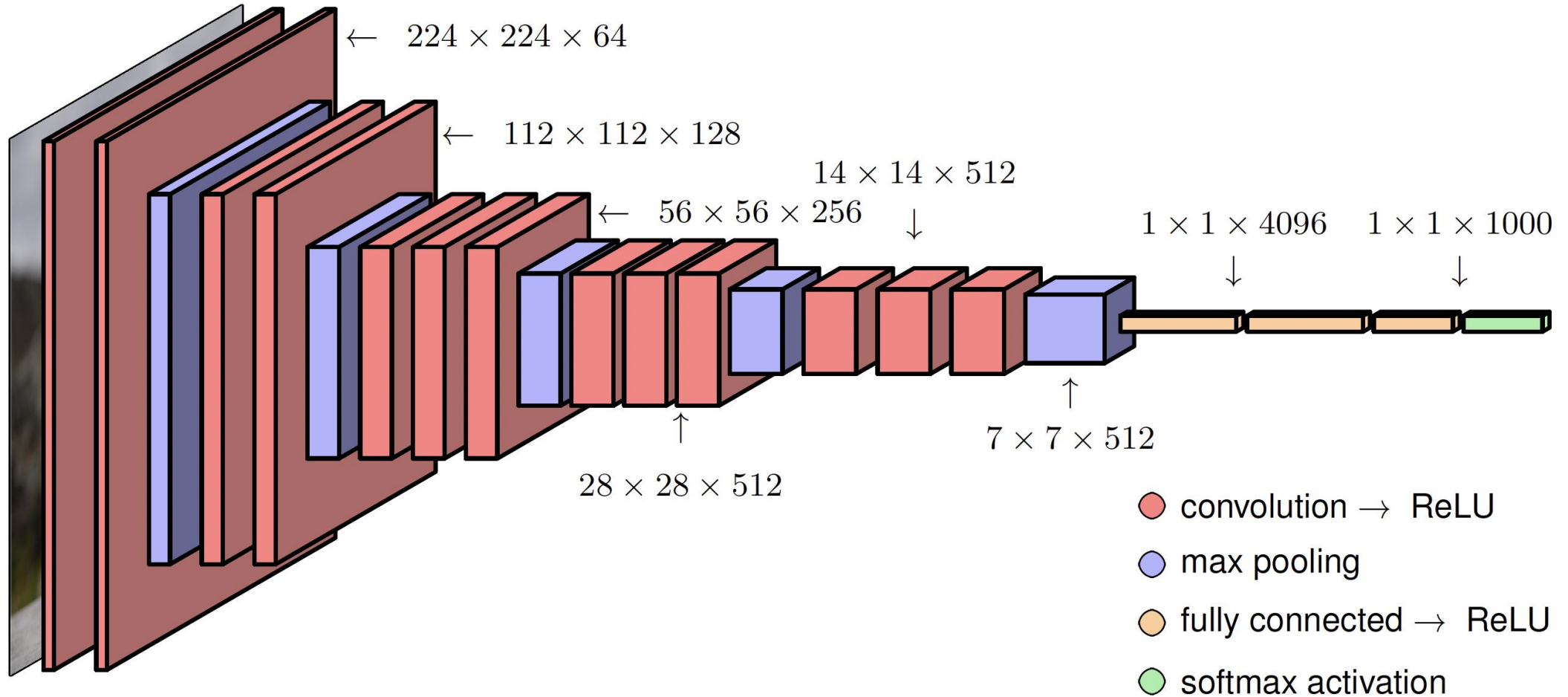
VGG



- The total number of parameters in all layers was about 138,000,000
- Nearly 75% of the parameters are in a single layer of connections. The majority of the remaining parameters are in the final two fully connected layers
- Dense connectivity accounts for 90% of the parameter footprint in the neural network

input image

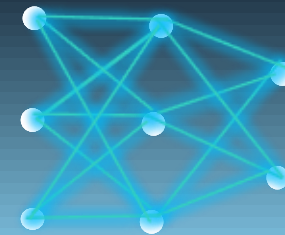
$224 \times 224 \times 3$





Number of Layers in Top Performing ILSVRC Contest Entries

Name	Year	Number of Layers	Top-5 Error
-	Before 2012	≤ 5	$> 25\%$
<i>AlexNet</i>	2012	8	15.4%
<i>ZfNet/Clarifai</i>	2013	8 / > 8	14.8% / 11.1%
<i>VGG</i>	2014	19	7.3%
<i>GoogLeNet</i>	2014	22	6.7%
<i>ResNet</i>	2015	152	3.6%



Thank you

